

Lecture 12: Space Complexity & Selection Sort

PROBLEM STATEMENT

In this lecture, we will understand **Space Complexity** deeply — how to calculate it, what auxiliary space is, and go through examples to visualize it. After that, we will run the **Selection Sort** algorithm step-by-step with a real array example, complete C++ code, and complexity analysis.

INTUITION

Space Complexity means — algorithm solve karne ke liye **extra kitni memory** chahiye. Input array ki memory nahi ginte, sirf extra variables, arrays, aur data structures ki memory count hoti hai. **Auxiliary Space** woh extra space hoti hai jo algorithm ke execution ke dauran use hoti hai.



Part 1: Space Complexity Deep Dive

What is Space Complexity?

Space Complexity = Input Space + Auxiliary Space. Lekin generally jab hum Space Complexity bolte hain, toh **Auxiliary Space** ki baat karte hain — matlab algorithm ke andar jo extra space use ho rahi hai.

WORKFLOW / ALGORITHM

Space Complexity Calculate Karne Ka Tarika:

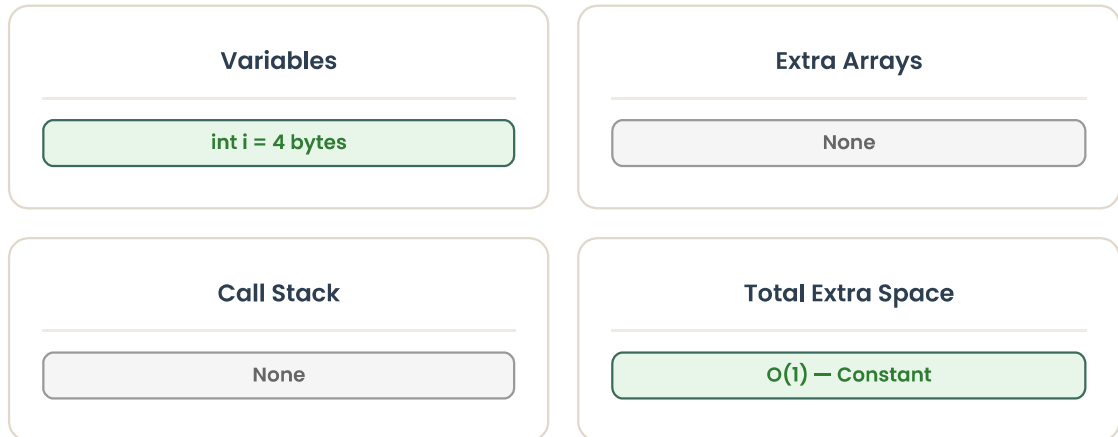
1. Code mein dekho ki **kitne extra variables** declare ho rahe hain.
2. Agar **new array** bana rahe ho toh uski size count karo.
3. Recursion mein **call stack** ki memory bhi add karo.
4. Input array ki memory **ignore** karo — woh toh already given hai.
5. Sabse bade extra space wale term ko pakad lo — wahi S.C. hoti hai.

Example 1: Simple Loop ($O(1)$ Space)

```
// Only one variable 'i' is using extra space
for (int i = 0; i < n; i++) {
    cout << i;
}
```

DRY RUN / VISUALIZATION

Memory usage breakdown:



Sirf ek `int i` variable hai jo **4 bytes** leta hai. Chahe `n = 10` ho ya `n = 106`, space same rehti hai. Isliye **S.C. = $O(1)$** .

Example 2: Reverse Array with Extra Array ($O(n)$ Space)

```
// Input array
int arr[] = {10, 20, 30, 40};

// Extra array liya — yahi space complexity badhata hai
int newarr[n];

int j = n - 1;
for (int i = 0; i < n; i++) {
    newarr[j] = arr[i];
    j--;
}
return newarr;
```

DRY RUN / VISUALIZATION

Array state after each iteration ($n = 4$):

Initial:



After $i=0, j=3$: $\text{newarr}[3] = 10$



After $i=1, j=2$: $\text{newarr}[2] = 20$



After $i=2, j=1$: $\text{newarr}[1] = 30$



After $i=3, j=0$: $\text{newarr}[0] = 40 \rightarrow$ Final:



COMPLEXITY ANALYSIS

Extra Space Used:

- `newarr[n]` $\rightarrow$ n integers
- `int i, int j` $\rightarrow$ 2 variables

Result:

Dominant term: n

S.C. = $O(n)$

$\Omega(n)$

$\Theta(n)$

Key Insight: Jab bhi tum **naya array banate ho** input ke size ka, toh Space Complexity $O(n)$ ho jaati hai. Agar tum **in-place** (same array mein) swap karoge, toh sirf $O(1)$ extra space lagegi.

Example 3: In-Place Reverse ($O(1)$ Space)

```
// In-place reverse – no extra array needed
int i = 0, j = n - 1;
while (i < j) {
    swap(arr[i], arr[j]);
    i++;
    j--;
}
```

BEST PRACTICE / OPTIMIZATION

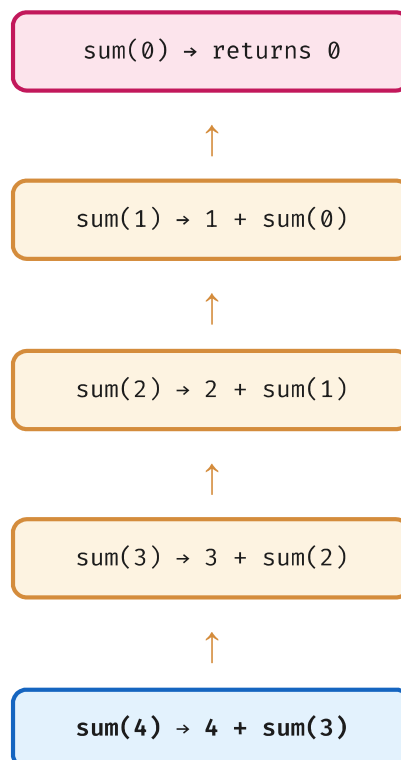
Yeh approach **Two Pointer Technique** use karti hai. Sirf do variables `i` aur `j` extra use ho rahe hain — chahe array kitna bada ho. Isliye **S.C. = $O(1)$** . Hamesha try karo ki **in-place** solve karo, extra array mat banao.

Example 4: Recursive Sum ($O(n)$ Space — Call Stack)

```
// Recursive function – each call uses stack space
int sum(int n) {
    if (n == 0) return 0;
    return n + sum(n - 1);
}
```

DRY RUN / VISUALIZATION

Call Stack for `sum(4)` :



Har recursive call stack mein ek frame add hoti hai. `n = 4` ke liye 5 frames. `n` ke liye `n+1` frames. Isliye **S.C. = $O(n)$** (call stack space).

Space Complexity Cheat Sheet

Scenario	Extra Space Used	Space Complexity
Few variables only	int, float variables	$O(1)$
New array of size n	int newarr[n]	$O(n)$
2D matrix n×n	int matrix[n][n]	$O(n^2)$
Recursion depth n	Call stack frames	$O(n)$
Binary recursion	Tree of calls	$O(n)$ to $O(\log n)$



Part 2: Selection Sort Algorithm

Real-World Analogy

Socho class mein **5 students** line mein khade hain unki height ke hisaab se: [10, 3, 9, 9, 5]. Teacher ko sabse **chhote student ko sabse aage** laana hai. Phir bache hue mein se phir sabse chhote ko second position pe, and so on. Yahi logic **Selection Sort** ka hai — har step pe **minimum element dhoondh ke usko correct position pe lao**.

PROBLEM STATEMENT

Given array [10, 3, 9, 9, 5] ko **ascending order** mein sort karna hai using Selection Sort. Step-by-step dry run karo, code likho, aur complexity analyze karo.

Algorithm Steps

WORKFLOW / ALGORITHM

- Outer loop $i = 0$ se $n-2$ tak chalega.
- Har i ke liye, assume karo ki i wala index **minimum** hai.
- Inner loop $j = i+1$ se $n-1$ tak — **remaining array mein minimum dhoondo**.
- Agar $nums[j] < nums[\text{minIndex}]$, toh $\text{minIndex} = j$.
- Inner loop ke baad, $\text{swap}(nums[i], nums[\text{minIndex}])$.
- Repeat until sorted.

Complete C++ Implementation

```

class Solution {
public:
    vector<int> sortArray(vector<int>& nums) {
        int n = nums.size();

        // Selection Sort
        for (int i = 0; i < n - 1; i++) {

            // Assume current index has the smallest element
            int minIndex = i;

            for (int j = i + 1; j < n; j++) {
                if (nums[j] < nums[minIndex]) {
                    minIndex = j;
                }
            }

            // Swap current element with minimum found
            swap(nums[i], nums[minIndex]);
        }

        return nums;
    }
};

```

Step-by-Step Visual Dry Run

Input Array: [10, 3, 9, 9, 5]

1 Pass 1: i = 0, Find minimum in [10, 3, 9, 9, 5]

minIndex start = 0 (value 10). Inner loop checks j = 1 to 4:



Minimum = 3 at index 1. Swap index 0 and index 1.



Array after Pass 1: [3, 10, 9, 9, 5] → 3 is now sorted at position 0 ✓

2 Pass 2: i = 1, Find minimum in [10, 9, 9, 5]

minIndex start = 1 (value 10). Inner loop checks j = 2 to 4:



Minimum = 5 at index 4. Swap index 1 and index 4.



Array after Pass 2: [3, 5, 9, 9, 10] → 3, 5 sorted ✓

3 Pass 3: i = 2, Find minimum in [9, 9, 10]

minIndex start = 2 (value 9). Inner loop checks j = 3 to 4:



Minimum = 9 at index 3. But 9 is already at index 2 (same value). Swap index 2 and 3 — no visual change.



Array after Pass 3: [3, 5, 9, 9, 10] → 3, 5, 9 sorted ✓

4 Pass 4: i = 3, Find minimum in [9, 10]

minIndex start = 3 (value 9). Inner loop checks j = 4:



Minimum = 9 at index 3 (already minimum). No swap needed.

Array after Pass 4: [3, 5, 9, 9, 10] → Fully Sorted ✓✓✓

DRY RUN / VISUALIZATION — SUMMARY TABLE

Pass	i	Subarray Scanned	Min Found	Swap With	Array After Pass
1	0	[10, 3, 9, 9, 5]	3 @ idx 1	idx 0	[3, 10, 9, 9, 5]
2	1	[10, 9, 9, 5]	5 @ idx 4	idx 1	[3, 5, 9, 9, 10]
3	2	[9, 9, 10]	9 @ idx 3	idx 2	[3, 5, 9, 9, 10]
4	3	[9, 10]	9 @ idx 3	—	[3, 5, 9, 9, 10]

Time & Space Complexity Analysis

COMPLEXITY ANALYSIS

Time Complexity:

Outer loop: $n-1$ iterations
Inner loop: $(n-1) + (n-2) + \dots + 1$
Sum = $n(n-1)/2 = (n^2 - n)/2$
T.C. = $O(n^2)$

Space Complexity:

Variables: $i, j, \text{minIndex}$
No extra array used
In-place sorting
S.C. = $O(1)$

Key Insight: Selection Sort mein *hamesha* $O(n^2)$ comparisons hote hain chahe array sorted ho ya ulta. Kyunki har pass mein inner loop poora chalta hai minimum dhoondhne ke liye. Isliye **Best Case = Worst Case = Average Case = $O(n^2)$** . Lekin **swaps sirf $O(n)$** hote hain — har pass mein maximum 1 swap.

BEST CASE

Already sorted array

$O(n^2)$

AVERAGE CASE

Random order array

$O(n^2)$

WORST CASE

Reverse sorted array

$O(n^2)$



Real-World Applications of Sorting

INTUITION

Sorting sirf DSA ka topic nahi hai — yeh **real life mein har jagah use hota hai** :

- **Myntra / Flipkart** — "Sort by Price: Low to High" — products ko price ke hisaab se arrange karna.
- **Amazon** — "Sort by Rating" — sabse zyada rated products pehle dikhana.
- **Google Maps** — "Sort by Distance" — sabse paas wale restaurants pehle.
- **Excel / Sheets** — Data ko alphabetical ya numerical order mein arrange karna.
- **Class Ranking** — Students ko marks ke hisaab se sort karna.

Har jagah same concept hai — **data ko ascending order mein arrange karna** taaki user ko easily samajh aa sake.



Big-O, Omega, Theta Revisited

O Big-O (Upper Bound) — Sabse Zyada

Big-O batata hai ki algorithm **maximum kitna time** lega. Agar T.C. = n^2 hai, toh $O(n^2)$, $O(n^3)$, $O(n^4)$ sab valid upper bounds hain. Lekin hum **tightest upper bound** dete hain — matlab $O(n^2)$.

Interview Rule: Hamesha Big-O batana. $O(n^2)$ ke jagah $O(n^3)$ mat likhna — interviewer ko pata chal jaata hai ki tumhe exact complexity nahi aati.

Ω Omega (Lower Bound) — Sabse Kam

Omega batata hai ki algorithm **minimum kitna time** lega. Agar T.C. = n^2 hai, toh $\Omega(n^2)$, $\Omega(n)$, $\Omega(1)$ sab valid lower bounds hain. Lekin tightest lower bound $\Omega(n^2)$ hota hai.

Θ Theta (Tight Bound) — Exact

Theta tab use hota hai jab **upper bound = lower bound**. Selection Sort ke case mein T.C. hamesha n^2 hai — chahe best case ho ya worst. Isliye $\Theta(n^2)$ bhi valid hai. Lekin practically sirf Big-O use karte hain.

IMPORTANT

Selection Sort mein teeno cases same hain — Best, Average, Worst sab $O(n^2)$. Kyunki chahe array sorted ho ya nahi, inner loop hamesha poora chalta hai minimum dhoondhne ke liye. Yeh **Bubble Sort se behtar** isliye hai kyunki swaps kam hote hain (maximum n swaps), lekin comparisons same hain.



Sorting Algorithms Comparison

Algorithm	Best Case	Average Case	Worst Case	Space	Stable?
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	No
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Yes
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes



Interview Tips & Common Pitfalls

INTERVIEW TIP

- Selection Sort ka dry run** zaroor practice karo — interview mein pen-paper pe dry run mangte hain.
- Space Complexity puchein toh **auxiliary space** batana — input array ki memory nahi ginte.
- Selection Sort **stable nahi hai** — equal elements ka original order change ho sakta hai.
- Swaps vs Comparisons alag hote hain — Selection Sort mein **comparisons $O(n^2)$** lekin **swaps $O(n)$** .
- Big-O mein hamesha **tightest bound** do — $O(n^2)$ ke jagah $O(n^3)$ mat likhna.

EDGE CASE

- $n = 0$ ya $n = 1$ — already sorted, loop nahi chalega.
- **Duplicate values** — Selection Sort duplicates ko handle kar leta hai lekin stable nahi hai.
- Already sorted array — phir bhi $O(n^2)$ comparisons honge (yeh disadvantage hai).
- Large arrays ke liye Selection Sort **avoid** karo — Merge Sort ya Quick Sort use karo.



Key Takeaways

TAKEAWAY

1. **Space Complexity** = Extra space jo algorithm use karta hai. Input space nahi ginte.
2. Simple variables → $O(1)$; New array → $O(n)$; Recursion → $O(n)$ stack space.
3. **In-place** algorithms (Two Pointer) hamesha behtar hote hain — extra memory bachati hai.
4. Selection Sort ka logic: **Har pass mein minimum dhoondh ke usko correct position pe lao** .
5. Selection Sort: T.C. = $O(n^2)$, S.C. = $O(1)$ — all cases same.
6. Swaps maximum $n-1$ hote hain — isliye write-heavy systems mein useful ho sakta hai.
7. Sorting real life mein har jagah — e-commerce, maps, databases, spreadsheets.
8. Big-O = Upper Bound (sabse zyada), Omega = Lower Bound (sabse kam), Theta = Tight Bound (exact).